

JSP 요청 위임과 모델2 게시판 구현

오늘의 강의 학습 요약

금일 강의에서는 서블릿과 JSP를 함께 사용하는 이유를 출발점으로, 로직(서블릿)과 화면(JSP)을 분리했을 때 요청을 어떻게 이어 붙이는지(요청 위임)와 그 핵심인 forward/redirect의 차이를 정리했습니다. 이어서 HTTP 프로토콜의 연결/상태 특성(커넥션리스, 스테이트리스) 때문에 세션과 쿠키가 왜 필요한지, 그리고 세션/쿠키가 상태를 유지하는 방식의 차이를 설명했습니다.

후반부에는 JSP의 실행 원리(변환→컴파일→실행)와 변환 결과물인 .java(서블릿 유사 코드)를 워크 디렉터리에서 직접 확인하며, JSP의 내장 객체(6개)와 JSP 태그(디렉티브, 스크립트 요소, 주석 등)를 코드 변환 관점에서 매핑했습니다. 마지막으로 모델2(MVC) 아키텍처로 MyBatis 기반 게시판 CRUD(목록/작성/상세/조회/수정/삭제)를 단계적으로 구현하며, 스코프 저장과 포워드/리다이렉트 선택 기준을 실습으로 연결했습니다.

1. 서블릿, JSP 분리와 요청 위임

서블릿은 비즈니스 로직, JSP는 화면 처리를 담당하도록 분리하며, 둘을 연결하기 위해 요청 위임 (Forward/Redirect)을 사용합니다.

서블릿 코드에는 자바 로직과 HTML 출력이 섞이기 쉬우며, 이 경우 자바 개발자와 화면 담당자가 같은 파일을 함께 수정해야 하므로 유지보수 위험이 커집니다. 로직과 뷰가 혼재하면 한쪽 수정이 다른 쪽(화면/로직)을 깨뜨릴 수 있어 분리가 필요합니다.

분리 전략은 로직은 서블릿이, 화면은 JSP가 담당하도록 역할을 나누는 방식입니다. 다만 서블릿이 직접 HTML로 응답하던 내용을 JSP로 옮기면, 서블릿에서 JSP로 “요청을 넘기는” 연결이 필요하며 이를 요청 위임이라고 정리됩니다.

포워드(Forward)는 서블릿 요청 시 만들어진 동일한 `HttpServletRequest` 를 JSP까지 그대로 전달합니다. 따라서 브라우저 URL이 변경되지 않으며, 서블릿에서 `request.setAttribute()` 로 저장한 요청 스코프 데이터가 JSP에서 그대로 조회됩니다.

리다이렉트(Redirect)는 서블릿이 브라우저에 “다른 URL로 다시 요청하라”는 응답을 주는 방식입니다. 결과적으로 요청이 2번 생성되며 URL이 변경됩니다. 요청 객체가 새로 만들어지므로 기존 요청 스코프 데이터는 JSP에서 접근할 수 없습니다.

구분	Forward	Redirect
요청(Request) 생성	1회입니다.	2회입니다.
URL 변경	변경되지 않습니다.	변경됩니다.
request 스코프 공유	가능합니다.	불가능합니다.
주요 용도	데이터 전달이 필요한 화면 이동에 적합합니다.	새 요청이 필요한 이동, 중복 제출 방지 등에 적합합니다.

핵심 키워드 #요청위임 #Forward #Redirect #request스코프 #모델2

2. HTTP 특성과 세션, 쿠키

HTTP는 커넥션리스, 스테이트리스 특성을 가지므로, 로그인, 장바구니 같은 상태 유지를 위해 세션과 쿠키를 사용합니다.

HTTP는 다수의 클라이언트가 서버에 동시 접속하는 환경에서 연결을 계속 유지하기 어렵기 때문에, 요청-응답이 끝나면 연결을 끊는 커넥션리스 방식으로 동작합니다. 연결을 끊어버리면 서버가 클라이언트의 상태를 지속적으로 기억하기 어렵기 때문에 스테이트리스(상태 비저장) 특성을 갖습니다.

그러나 웹 애플리케이션에서는 이전 작업을 다음 화면에서 이어 받아야 하는 경우가 많습니다. 대표적으로 로그인 여부, 장바구니 담기 정보 등은 다음 요청에서도 필요합니다. 이를 해결하기 위해 상태를 어디엔가 저장해야 하며, 저장 위치는 서버 또는 클라이언트로 나뉩니다.

서버에 상태를 저장하는 방식이 세션(Session)이며, 자바 기반 환경에서는 다양한 타입의 객체를 저장할 수 있어 활용 폭이 넓습니다. 반면 클라이언트에 저장하는 방식이 쿠키(Cookie)이며, 요청마다 자동 전송되는 특성이 있지만 사용자가 쿠키 저장을 거부할 수 있고 저장 형태/보안 측면에서 제약이 큼니다.

구분	세션(Session)	쿠키(Cookie)
저장 위치	서버입니다.	클라이언트입니다.
데이터 성격	자바 객체 등 다양한 데이터가 가능합니다.	제한적 형태로 저장됩니다.
전송 방식	세션 식별자를 기반으로 서버가 매핑합니다.	요청마다 자동으로 함께 전송됩니다.
특징	서버가 상태를 관리하므로 제어가 용이합니다.	사용자가 거부할 수 있어 리스크가 있습니다.

핵심 키워드 #HTTP #커넥션리스 #스테이트리스 #세션 #쿠키

3. JSP 실행 원리와 변환된 자바 코드

JSP는 변환→컴파일→실행 순서로 처리되며, 변환 결과인 .java는 서블릿과 유사한 구조를 가집니다.

JSP는 곧바로 실행되는 것이 아니라, 먼저 자바 소스(.java)로 변환되고 컴파일되어 클래스가 된 뒤 실행됩니다. 이 변환 결과물은 “거의 서블릿”이라고 볼 수 있을 정도로 유사하며, JSP를 서블릿으로 이해하면 표현법만 다를 뿐 같은 일을 한다는 관점이 정리됩니다.

변환된 JSP 자바 파일은 톰캣 `work` 디렉터리 아래에 생성됩니다. 또한 JSP는 “요청되어야” 변환이 발생하므로, 파일을 만들어도 요청하지 않으면 변환 산출물이 생기지 않습니다.

JSP의 변환 결과를 보면, 서블릿의 `doGet/doPost` 역할에 대응되는 `_jspService()`가 있으며, 이 메소드 내부에 HTML 출력과 스크립틀릿 코드가 배치됩니다. 따라서 JSP에서 `<% ... %>`로 작성한 자바 코드는 `_jspService()` 내부(서블릿의 `doGet/doPost` 내부와 같은 위치)로 들어간다고 이해하시면 됩니다.

JSP 표현	변환 관점에서의 의미	서블릿 관점 대응
HTML 태그	출력 문자열로 변환됩니다.	<code>out.print()</code> 호출 흐름입니다.
<code><% ... %></code>	<code>_jspService()</code> 내부 자바 코드입니다.	<code>doGet/doPost</code> 내부 자바 코드입니다.
<code><%= ... %></code>	출력 코드로 변환됩니다.	<code>out.print(표현식)</code> 입니다.

핵심 키워드

#JSP변환

#_jspService

#톰캣work

#스크립틀릿

#익스프레션

4. JSP 내장 객체(내장 변수) 6개

JSP는 변환 시점에 6개 내장 객체가 자동 선언되며, 선언 없이 정해진 이름으로만 사용해야 합니다.

JSP 변환 산출물에는 특정 객체들이 자동으로 선언됩니다. 이 객체들은 `_jspService()` 내부에서 생성되므로, 스크립틀릿(`<% ... %>`)에서 선언 없이 사용할 수 있습니다. 다만 개발자가 만드는 서블릿과 달리, JSP 내장 객체의 변수명은 고정이므로 다른 이름으로 바꾸어 사용할 수 없습니다.

내장 객체는 최소 6개를 핵심으로 기억해야 하며, 이름 자체가 고정된 약속입니다. 또한 스코프 개념(요청/세션/애플리케이션)과 연결되어, 어떤 범위에 데이터를 저장할지 결정할 때 중요한 기반이 됩니다.

내장 객체 변수명	실체 타입(개념)	주 용도
<code>request</code>	<code>HttpServletRequest</code>	요청 파라미터/요청 스코프 데이터 접근입니다.
<code>response</code>	<code>HttpServletResponse</code>	응답 제어입니다.
<code>session</code>	<code>HttpSession</code>	세션 스코프 상태 관리입니다.
<code>application</code>	<code>ServletContext</code>	애플리케이션 스코프 공유 데이터입니다.
<code>config</code>	<code>ServletConfig</code>	서블릿 설정 접근입니다.
<code>out</code>	<code>JspWriter</code>	화면 출력(서블릿의 <code>PrintWriter</code> 유사)입니다.

핵심 키워드

#내장객체

#request

#session

#application

#JspWriter

5. JSP 디렉티브: page / include / taglib

디렉티브는 JSP에게 정보를 지시하며, page 디렉티브의 contentType, import가 핵심이고 include는 화면 재사용을 제공합니다.

디렉티브는 JSP에게 특정 정보를 전달하는 지시자 태그입니다. 모델2에서는 JSP가 화면 역할에 집중하므로 모든 속성을 알 필요는 없고, 실무적으로 중요한 항목 위주로 정리됩니다.

page 디렉티브의 contentType 은 서버릿의 response.setContentType() 에 대응되며, 브라우저가 응답을 HTML로 렌더링할지 텍스트로 표시할지 결정합니다. charset 은 한글 깨짐 이슈가 있을 때 중요한 옵션이며, 실습에서는 text/plain 으로 바꾸면 태그가 렌더링되지 않고 그대로 출력되는 것을 확인했습니다.

page 디렉티브의 import 는 JSP 내부에서 자바 클래스를 사용하기 위한 импорт 선언입니다. JSP에서는 자동 импорт 단축키가 제한될 수 있으므로, Ctrl+Space 자동완성으로 import 구문을 생성하는 방식이 유용하다고 정리되었습니다.

include 디렉티브는 화면 재사용을 위한 포함 기능이며, 같은 화면(헤더/푸터/메뉴 등)을 여러 페이지에서 반복 작성하지 않고 포함시킬 때 사용합니다. 또한 include에는 디렉티브 방식과 액션 태그 방식이 있으며 동작 방식이 다릅니다.

구분	디렉티브 include (<%@ include file="..." %>)	액션 include (<jsp:include page="..." />)
포함 시점	JSP 변환 전, 정적으로 포함됩니다.	실행 시, 요청하여 결과를 포함합니다.
포함 대상 JSP 변환	포함 파일이 별도 .java로 생성되지 않을 수 있습니다.	포함 파일도 요청되어 변환/실행 흐름을 가질 수 있습니다.
포함되는 것	코드 자체가 합쳐집니다.	실행 결과(출력)가 포함됩니다.

액션 include 사용 시 flush="true" 를 관례적으로 부여하는 이유도 설명되었습니다. 큰 응답을 한 번에 전송하지 않고 버퍼가 찼을 때 전송되도록 하여, 대용량 출력에서 끊김/응답 실패 위험을 줄이는 목적입니다.

taglib 디렉티브는 JSTL에서 태그 라이브러리를 사용할 때 필요하며, 강의에서는 JSTL 파트에서 상세히 다루기로 하고 문법 형태만 개괄했습니다.

핵심 키워드

#page디렉티브

#contentType

#import

#include

#jsp:include

6. JSP 스크립트 요소와 주석

디클레이션, 스크립틀릿, 익스프레션은 각각 생성 위치와 목적이 다르며, JSP 주석은 HTML 주석과 구분해야 합니다.

디클레이션(`<%! ... %>`)은 서블릿의 인스턴스 영역(인스턴스 변수, `doGet/doPost` 외 메소드)에 해당하는 멤버를 선언할 때 사용합니다. 변환된 자바 파일에서 클래스 멤버로 생성되는 위치를 확인했으며, 일반적인 화면 중심 JSP에서는 사용 빈도가 낮다고 정리되었습니다.

스크립틀릿(`<% ... %>`)은 가장 많이 쓰는 형태로, 변환 시 `_jspService()` 내부로 들어갑니다. 즉 서블릿의 `doGet/doPost` 내부에서 하던 변수 선언, 로직 처리, 반복문/조건문 등을 JSP에서는 스크립틀릿으로 표현합니다.

익스프레션(`<%= ... %>`)은 브라우저 출력용이며, 변환 시 `out.print(...)` 형태로 들어갑니다. 변수뿐 아니라 반환값이 있는 메소드 호출 등 표현식 전반이 가능하며, 스크립틀릿과 섞어 반복문 내부에서 값을 출력하는 형태(스크립틀릿으로 블록을 열고 닫으며 중간에 익스프레션을 끼우는 방식)도 실습했습니다.

조건문/반복문을 JSP에서 사용할 때는 스크립틀릿 블록이 HTML 사이에서 분리되는 형태가 되며, 이는 변환된 자바 코드 관점에서 보면 자연스러운 구조임을 강조했습니다.

JSP 주석은 `<%-- ... --%>` 를 사용해야 하며, `<!-- ... -->` 는 HTML 주석이므로 JSP 코드 제거 목적과 다르게 동작할 수 있어 구분이 필요합니다. 또한 이클립스 단축키로 JSP 주석 블록 처리(`Ctrl+Shift+ /`)가 가능함을 실습했습니다.

요소	문법	변환 위치	주 용도
디클레이션	<code><%! ... %></code>	클래스 멤버 영역	인스턴스 변수/메소드 선언입니다.
스크립틀릿	<code><% ... %></code>	<code>_jspService()</code> 내부	로직, 제어문, 변수 선언입니다.
익스프레션	<code><%= ... %></code>	출력 코드	화면 출력입니다.
JSP 주석	<code><%-- ... --%></code>	변환 단계에서 제거	서버 측 주석 처리입니다.

핵심 키워드

#디클레이션

#스크립틀릿

#익스프레션

#JSP 주석

#제어문

7. 모델2(MVC) 게시판: 구조와 공통 패턴

모델2는 요청은 서블릿, 응답은 JSP로 처리하며, Controller-Model-View 분리와 MyBatis 계층 (Controller→Service→DAO→DB)을 적용합니다.

게시판은 웹 애플리케이션의 기본 CRUD 패턴을 가장 잘 담는 예제로, 구현 흐름 자체를 익숙하게 만드는 것이 중요하다고 강조되었습니다. 모델2에서는 브라우저 요청이 JSP로 직접 가지 않고 반드시 서블릿(컨트롤러)로 들어가야 하며, 서블릿이 서비스/DAO를 통해 DB를 처리한 뒤 JSP로 포워드하여 화면을 렌더링합니다.

구성 요소 역할은 다음과 같이 정리됩니다.

- 컨트롤러(Controller)는 서블릿이며, 요청을 받고 스코프에 데이터를 저장한 뒤 뷰로 위임합니다.
- 모델(Model)은 Service/DAO/DTO 및 DB 접근 계층이며, 비즈니스 로직과 데이터 처리를 담당합니다.
- 뷰(View)는 JSP이며, 화면 렌더링을 담당합니다.

또한 MyBatis 기반 구현에서 서비스 계층의 공통 패턴이 반복됨을 강조했습니다. 특히 `SqlSession` 획득, `try/finally` 로 close 처리, DML 수행 시 commit 수행이 반복됩니다.

계층	책임	반복되는 핵심 패턴
Servlet	파라미터 수집, 스코프 저장, 포워드/리다이렉트 결정	request 파라미터 → 서비스 호출 → setAttribute/redirect 입니다.
Service	세션 관리 및 트랜잭션(커밋) 제어	세션 획득 → DAO 호출 → DML이면 commit → close 입니다.
DAO	Mapper 호출로 DB 접근	<code>selectList/selectOne/insert/update/delete</code> 호출 입니다.
JSP	출력 렌더링	request 스코프에서 꺼내 반복/출력입니다.

핵심 키워드

#모델2

#MVC

#Service

#DAO

#MyBatis

8. 게시판 DB 테이블과 DTO 설계

게시판 테이블을 생성하고, DTO에 컬럼을 매핑하여 목록/상세/수정/삭제의 데이터 전달 단위를 표준화합니다.

테이블은 글번호(PK, auto increment), 제목, 작성자, 내용, 작성일(default current date), 조회수(default 0)로 구성했습니다. 초기 데이터 1건을 insert하여 목록/상세 기능 테스트 기반을 만들었습니다.

DTO는 테이블 컬럼과 1:1로 매핑하며, 작성일은 단순 출력 목적이면 String으로 처리할 수 있고, 연산/가공이 필요하면 LocalDateTime 등으로 설계하는 것이 바람직하다는 선택 기준을 제시했습니다.

컬럼	타입 예시	DTO 필드 예시
num	int, PK	int num
title	varchar	String title
author	varchar	String author
content	varchar/text	String content
writeDay	date	String writeDay 또는 날짜 타입
readCnt	int	int readCnt

핵심 키워드

#CREATE TABLE #BoardDTO #PK #writeDay #readCnt

9. 목록(List): request 스코프 + Forward

목록 조회는 단발성 데이터이므로 request 스코프에 저장하고 forward로 JSP에 전달하여 출력합니다.

목록 조회 서블릿은 서비스→DAO→MyBatis mapper를 통해 전체 레코드를 `List<BoardDTO>` 로 조회합니다. 조회 결과는 JSP에서 출력해야 하므로 서블릿에서 `request.setAttribute("list", list)` 로 저장합니다.

request 스코프 데이터를 JSP에서 사용해야 하므로, redirect는 사용할 수 없고 forward를 사용해야 합니다. forward는 동일 request를 공유하므로 JSP에서 `request.getAttribute("list")` 로 꺼내 출력할 수 있습니다.

JSP에서는 스크립틀릿으로 리스트를 꺼내 형변환하고, for-each 로 반복하여 `<tr>` 을 생성합니다. 이 과정에서 `<%= dto.getXxx() %>` 로 값 출력이 매핑된다는 점을 실습으로 확인했습니다.

핵심 키워드

#BoardListServlet

#request.setAttribute

#forward

#List<BoardDTO>

#테이블출력

10. 글 작성 UI: 모델2 원칙에 따른 화면 진입

글 작성 화면도 JSP 직접 요청이 아니라, UI용 서블릿을 통해 진입하여 모델2 규칙을 유지합니다.

글 작성 화면은 DB 연동이 없더라도, 모델2 규칙에 따라 브라우저는 UI용 서블릿을 요청하고 서블릿이 JSP로 이동시키는 방식을 사용합니다. 목록 화면 하단에 “글쓰기” 링크를 두고 UI 서블릿으로 이동시킨 뒤, 글쓰기 JSP를 보여주는 흐름을 구성했습니다.

UI 서블릿에서 JSP로 이동은 forward/redirect 모두 가능하지만, 화면만 보여주는 용도라면 redirect로도 구현 가능하다고 정리되었습니다. 다만 프로젝트 전반에서 “요청은 서블릿으로”라는 원칙을 일관되게 적용하는 것이 핵심입니다.

핵심 키워드

#WriteUI

#모델2규칙

#서블릿진입

#글쓰기폼

#링크

11. 글 작성(Create): DML 후 Redirect to 목록 서블릿

작성은 DML(insert)이므로 커밋이 필요하며, 완료 후에는 JSP가 아니라 목록 서블릿으로 리다이렉트 하여 최신 목록을 재조회합니다.

글 작성 서블릿은 폼에서 전달된 `title/author/content` 를 `request.getParameter()` 로 읽고 DTO에 담아 서비스에 전달합니다. MyBatis insert는 DML이므로 반환값은 보통 `int` (처리된 행 수)이며, 서비스에서 커밋을 수행합니다.

작성 완료 후 목록 화면으로 이동할 때, 목록 JSP로 직접 이동하면 "요청은 서블릿" 원칙이 깨지며 최신 목록 재조회도 누락될 수 있습니다. 따라서 `response.sendRedirect("BoardListServlet")` 로 목록 서블릿에 재요청하도록 구성합니다.

단계	핵심 처리
Servlet	파라미터 수집 → DTO 구성 → <code>service.write(dto)</code> → <code>redirect(listServlet)</code> 입니다.
Service	세션 획득 → <code>dao.write(session, dto)</code> → <code>commit</code> → <code>close</code> 입니다.
DAO/Mapper	insert 실행입니다.

핵심 키워드

`#insert`

`#commit`

`#sendRedirect`

`#BoardWriteServlet`

`#재요청`

12. 상세(Retrieve): 쿼리스트링 + request 스코프 + Forward

제목 링크에 글번호(num)를 쿼리스트링으로 전달하고, 해당 레코드를 조회해 request 스코프에 담아 forward로 상세 JSP에 출력합니다.

목록에서 제목을 클릭하면 글번호를 쿼리스트링으로 전달해야 하며, 글번호는 유니크한 PK이므로 조회 기준으로 적합합니다. URL은 `...?num=4` 형태로 구성하며, 상세 서블릿은 `request.getParameter("num")` 을 받아 `Integer.parseInt()` 로 변환합니다.

상세 조회 결과 `BoardDTO` 는 단발성 출력 데이터이므로 request 스코프에 저장하고 forward로 상세 JSP에 전달합니다. 상세 JSP는 `request.getAttribute()` 로 DTO를 꺼내 input value 및 textarea 본문에 출력합니다.

textarea는 input과 달리 `value` 속성이 아니라 태그 바디에 값을 넣어야 화면에 표시된다는 점을 함께 정리했습니다.

핵심 키워드

#쿼리스트링

#selectOne

#Integer.parseInt

#forward

#textarea출력

13. 조회수 증가(Update readCnt): 서비스에서 DML+조회 결합

상세 진입 시 조회수를 업데이트한 뒤 재조회하여 최신 readCnt를 화면에 표시하며, DML에는 커밋이 필요합니다.

조회수 증가는 `update board set readCnt = readCnt + 1 where num = ?` 형태의 업데이트 문으로 처리합니다. 상세 서비스 로직에서 먼저 `dao.readCnt(session, num)` 을 호출해 증가시키고, DML이므로 커밋한 뒤 `dao.retrieve(session, num)` 으로 데이터를 다시 조회합니다.

이렇게 하면 상세 화면에서 증가된 조회수가 반영된 DTO를 출력할 수 있습니다. 또한 DML이 반환하는 `int` 는 수정된 행 수이므로, 필요 시 `n == 1` 일 때만 commit 같은 조건 처리도 가능하다고 정리되었습니다.

핵심 키워드

#readCnt

#update

#commit조건

#서비스결합

#selectOne

14. 수정(Update): hidden 필드로 PK 전달 + DML 후 Redirect

수정은 title/author/content와 함께 PK(num)가 반드시 전달되어야 하며, 화면에 보이지 않게 전달할 때 hidden input을 사용합니다.

수정은 “어떤 글을 수정하는지”를 식별해야 하므로 글번호(num)가 함께 전달되어야 합니다. 제목/작성자/내용은 input/textarea로 제출되지만, 글번호는 화면에 텍스트로만 보여주면 submit에 포함되지 않으므로 input으로 포함시켜야 합니다.

글번호를 사용자가 수정하지 못하게 하면서 제출에 포함시키려면 `type="hidden"` 을 사용합니다. 대안으로 `disabled` 를 사용할 수 있으나, 일반적으로 hidden이 많이 사용된다고 정리되었습니다.

업데이트 매퍼는 DTO를 파라미터로 받아 `set title=?, author=?, content=? where num=?` 형태로 구성합니다. 업데이트 완료 후에는 목록 JSP가 아니라 목록 서블릿로 리다이렉트하여 최신 상태를 다시 조회하도록 구성합니다.

핵심 키워드

`#hidden`

`#update`

`#wherePK`

`#sendRedirect`

`#BoardUpdateServlet`

15. 삭제(Delete): 목록에서 링크로 PK 전달 + DML 후 Redirect

삭제는 목록에서 개별 삭제 링크를 제공하고, 쿼리스트링으로 num을 전달하여 삭제 후 목록 서블릿으로 리다이렉트합니다.

삭제는 다양한 UX로 배치할 수 있으나, 강의에서는 목록에 “삭제” 컬럼을 추가해 각 행마다 삭제 링크를 제공하는 방식으로 구현했습니다. 링크는 쿼리스트링으로 num을 전달하며, 삭제 서블릿은 retrieve와 동일하게 num 파라미터를 받아 정수로 변환합니다.

MyBatis delete는 DML이므로 서비스에서 커밋이 필요합니다. 삭제 완료 후에는 목록 JSP로 가지 않고 목록 서블릿으로 redirect하여 최신 목록을 재조회합니다.

핵심 키워드

`#delete`

`#쿼리스트링`

`#commit`

`#BoardDeleteServlet`

`#목록재조회`

16. 스코프 선택과 포워드/리다이렉트 의사결정

데이터 전달이 필요한 경우 request 스코프+forward를, 단순 이동/재요청이 목적이면 redirect를 선택합니다.

서블릿에서 JSP로 데이터를 넘겨야 하면 request 스코프 저장이 필요하며, 이 경우 동일 request를 유지하는 forward가 필수입니다. 반면 저장/수정/삭제처럼 처리 후 목록을 다시 조회해야 하는 경우에는 redirect로 목록 서블릿을 재요청하는 방식이 자연스럽습니다.

스코프 선택은 “데이터 생존 기간”에 따라 결정합니다. 단발성 출력 데이터는 request, 브라우저가 닫히기 전까지 유지가 필요하면 session, 서버 종료까지 유지하면 application이 적합합니다.

상황	스코프	위임 방식
목록/상세 출력 데이터 전달	request	forward
작성/수정/삭제 후 목록으로 이동	스코프 저장 불필요	redirect(목록 서블릿 재요청)

핵심 키워드

#스코프

#request

#session

#forward선택

#redirect선택

17. 이클립스 프로젝트 Import(워크샵 제공 파일)

제공된 프로젝트는 워크스페이스 외부에 압축 해제 후, Import로 불러오며 Copy 옵션을 체크하면 원본과 분리된 복사본으로 작업됩니다.

워크샵 제공 파일은 이클립스 프로젝트 형태로 제공되며, 워크스페이스 내부에 바로 압축을 풀어도 자동으로 프로젝트가 잡히지 않습니다. 워크스페이스 외부의 임의 디렉터리에 압축을 풀고, 이클립스에서 Import로 불러오는 방식이 필요합니다.

Import 경로는 `General > Existing Projects into Workspace` 를 사용하며, “Copy projects into workspace” 옵션을 체크하면 원본 폴더와 연결되지 않고 워크스페이스로 복사되어 안전하게 실습할 수 있습니다.

핵심 키워드

#Import

#ExistingProjects

#워크스페이스

#Copy옵션

#워크샵